Face Recognition: A Comparative Analysis of SVM and MLP

Axel Omar Sanchez Peralta (UCID: 30145429) · Mariia Podgaietska (UCID: 30151330)

0. Download as HTML

Make sure to run all cells first so outputs are included in the PDF

In [ ]:
```
import subprocess
from google.colab import files, drive
import shutil

drive.mount('/content/drive')

shutil.copy("/content/drive/MyDrive/Colab Notebooks/face_recognition.ipynb",

subprocess.run(
    ["jupyter", "nbconvert", "--to", "html", "/content/face_recognition.ipyr
    capture_output=True, text=True
)

files.download("/content/face_recognition.html")
```

1. Introduction & Motivation

Face recognition is a fundamental problem in computer vision with applications ranging from biometric authentication to human–computer interaction. Given a probe image of an unknown individual, a recognition system must reliably match it against a gallery of known subjects. A task that is complicated by intra-class variability in illumination, pose, and expression (i.e., the same person can look quite different across photos due to lighting, angle, or facial expression).

This notebook presents a systematic comparison of two classical supervised learning paradigms on the **AT&T face database**: the **Support Vector Machine (SVM)** and the **Multi-Layer Perceptron (MLP)**. Both models are evaluated under two feature representations, compact mathematical summaries computed from the raw pixel images, before being passed to the classifier:

Feature	Description
LBP histogram	Local Binary Pattern texture descriptor with parameters P=12, R=3 (defined in Section 4.1)

Feature	Description
PCA projection	Principal Component Analysis subspace of dimension k=100 (defined in Section 4.3)

Project objectives:

1. Implement a reproducible, stratified train/test split to prevent *subject leakage*, a form of data contamination where images of the same person appear in both training and testing sets, artificially inflating performance.
2. Tune both models with `HalvingRandomSearchCV` for efficient hyperparameter search (the process of finding the best model settings automatically).
3. Compare performance quantitatively via accuracy, ROC AUC, and DET curves (all defined when introduced).
4. Provide interpretable guidance on when to prefer SVM over MLP and vice versa.

2. Setup

In [1]:
```
import os, zipfile, time
from pathlib import Path

import requests
import cv2
import numpy as np
from glob import glob

from skimage.feature import local_binary_pattern

from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.experimental import enable_halving_search_cv  # must precede ne
from sklearn.model_selection import HalvingRandomSearchCV
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_curve, auc, det_curve

import matplotlib.pyplot as plt
import matplotlib

matplotlib.rcParams["figure.dpi"] = 120
```

3. Dataset

The **AT&T Dataset** contains **400 grayscale images** of 40 distinct subjects, with 10 images per subject. Each image has a resolution of 92 × 112 pixels (width × height) at 8-bit depth, meaning each pixel's brightness is represented as an integer from 0 (black) to 255 (white). The collection exhibits controlled but realistic variability:

- **Lighting:** slight changes in illumination across sessions.
- **Expression:** open/closed eyes, smiling/non-smiling faces.
- **Facial details:** presence or absence of glasses.
- **Pose:** small rotations and tilts (up to 20°).

Stratified split strategy. We divide the dataset into a *training set* (used to fit the model) and a *test set* (used to evaluate it on unseen data) using scikit-learn's `train_test_split` with two key arguments:

- `test_size=0.2` — reserves 20% of the data for testing, leaving 80% for training. This gives 320 training images and 80 test images.
- `stratify=y` — ensures that the split is *stratified*, meaning each of the 40 subjects contributes the same proportion of images to both sets. Concretely, every subject contributes exactly 8 images to the training set and 2 images to the test set. Without stratification, a naïve random split might by chance place all images of some subjects into only one partition, making fair evaluation impossible.

3.1 Download Dataset Function

To ensure reproducibility we implement a function that downloads the zipped dataset from our repository and extracts it to a specified directory. This function checks if the dataset already exists locally to avoid redundant downloads.

In [2]:
```python
def download_data(source: str, destination: str, remove_source: bool = True)
    """Downloads a zipped dataset from source and unzips to destination.

    Args:
        source: URL of the zip file.
        destination: Target sub-directory inside data/.
        remove_source: Remove the zip after extraction.

    Returns:
        pathlib.Path to the extracted directory.
    """
    data_path = Path("data/")
    image_path = data_path / destination

    if image_path.is_dir():
        print(f"[INFO] {image_path} directory exists, skipping download.")
    else:
        print(f"[INFO] Creating {image_path}...")
        image_path.mkdir(parents=True, exist_ok=True)
        target_file = Path(source).name
        print(f"[INFO] Downloading {target_file}...")
        with requests.get(source, stream=True, timeout=30) as r:
            r.raise_for_status()
            with open(data_path / target_file, "wb") as f:
                for chunk in r.iter_content(chunk_size=8192):
                    if chunk:
```

```
                        f.write(chunk)
        with zipfile.ZipFile(data_path / target_file, "r") as zf:
            print(f"[INFO] Unzipping {target_file}...")
            zf.extractall(image_path)
        if remove_source:
            os.remove(data_path / target_file)
    return image_path
```

3.2 Resolve Dataset

The dataset is organized into a directory structure where each subject's images are stored in a separate folder named `sX` (where `X` is the subject ID from 1 to 40). Each folder contains 10 images named `Y.pgm` (where `Y` is the image number from 1 to 10).

In [3]:
```
dataset_dir = Path("data/dataset")
zip_path = Path("data/dataset.zip")

if dataset_dir.exists():
    print("[INFO] Dataset already available.")
elif zip_path.exists():
    print("[INFO] Unzipping local dataset.zip...")
    dataset_dir.mkdir(parents=True, exist_ok=True)
    with zipfile.ZipFile(zip_path, "r") as zf:
        zf.extractall(dataset_dir)
else:
    print("[INFO] Downloading dataset from GitHub...")
    download_data(
        source="https://github.com/Axeloooo/Face-Recognition/raw/devel/data/
        destination="dataset",
    )
```

```
[INFO] Downloading dataset from GitHub...
[INFO] Creating data/dataset...
[INFO] Downloading dataset.zip...
[INFO] Unzipping dataset.zip...
```

3.3 Get Dataset Function

The `get_data` function reads the images from the resolved dataset path, and returns the feature matrix `X` and label vector `y`.

In [4]:
```
DATA_PATH = "data/dataset/ATT dataset/s*/**/*.pgm"


def get_data(path):
    """Loads images and labels from the specified path.

    Args:
        path (str): Glob pattern to load images (e.g., 'data/dataset/ATT dat

    Returns:
        Tuple[np.ndarray, np.ndarray]: Tuple of (X, y) where X is the featur
    """
```

```
    paths = glob(path, recursive=True)
    images, labels = [], []

    for p in paths:
        img = cv2.imread(p, 0)
        parts = Path(p).parts
        subject_label = [x for x in parts if x.startswith("s") and x[1:].isd
        img = np.float32(np.array(img) / 255.0)
        images.append(img)
        labels.append(int(subject_label[0][1:]) - 1)

    print(
        f"[INFO] Loaded dataset with {len(images)} samples and {len(set(labe
    )

    return np.array(images), np.array(labels)
```

3.4 Load Dataset

We call the `get_data` function to load the dataset into memory, obtaining the raw pixel features and corresponding labels. The images are resized to a consistent shape and normalised to the [0, 1] range for better numerical stability during model training.

In [5]:
```
X_raw, y = get_data(DATA_PATH)
print(f"[INFO] Raw data shape: {X_raw.shape}, Labels shape: {y.shape}")
```

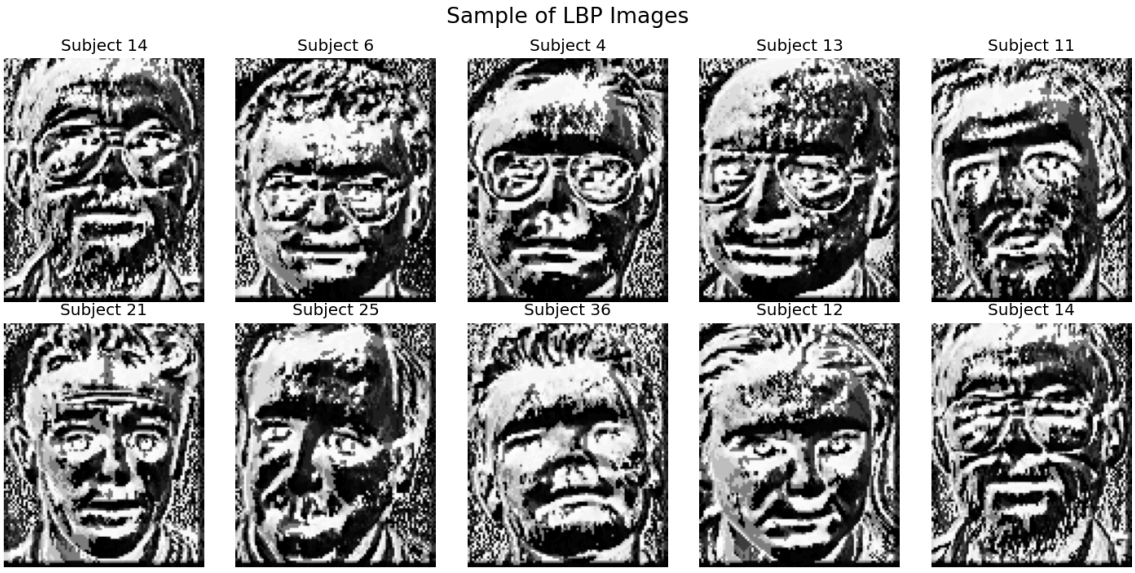
```
[INFO] Loaded dataset with 400 samples and 40 classes.
[INFO] Raw data shape: (400, 112, 92), Labels shape: (400,)
```

4. Feature Extraction

Raw pixel intensities are a natural baseline but are high-dimensional (92 × 112 = 10 304 features) and sensitive to illumination shifts. We therefore compare three representations that trade dimensionality for increasing degrees of invariance:

Representation	Dimension	Invariance
Raw pixels	10 304	None
LBP histogram	64	Monotone illumination
PCA projection	100	None (but low-rank)

4.1 Local Binary Patterns (LBP)



What is LBP?

Local Binary Pattern (LBP) is a non-parametric texture operator that encodes the local neighbourhood of each pixel as a binary string. "Non-parametric" means it makes no assumption about the underlying distribution of pixel values, it simply compares each pixel to its neighbours.

How it works (intuition):

For each pixel in the image, LBP looks at a circular ring of neighbouring pixels around it. It compares each neighbour's brightness to the centre pixel's brightness. If a neighbour is brighter than or equal to the centre, it is assigned a `1`; otherwise a `0`. Reading these bits around the circle in order produces a binary number (e.g., `10110011`), which is the LBP code for that pixel. This process is repeated for every pixel in the image, producing an *LBP map*, an image of the same size where each pixel holds its LBP code instead of its original brightness.

Parameters used — P=12, R=3:

- **P** is the number of neighbouring sample points on the circular ring. We use **P=12**, meaning 12 equally-spaced points are sampled around each centre pixel. A higher P captures more directional texture detail but increases computation.
- **R** is the radius of the circular neighbourhood in pixels. We use **R=3**, meaning neighbours are sampled on a circle of radius 3 pixels from the centre. A larger R captures coarser, more global texture patterns; a smaller R captures finer, more local ones. R=3 is a common choice that balances local detail with robustness to small spatial misalignments in face images.

Why these values? P=12 and R=3 are a well-established combination in face recognition literature. They provide a good trade-off: fine enough to capture distinctive

facial textures (wrinkles, edges, pores) while being robust enough to tolerate small variations in alignment and illumination.

From LBP map to feature vector:

Rather than feeding the entire LBP map (92×112 = 10,304 values) into the classifier, we summarise it as a *normalised histogram with 64 bins*. This histogram counts how often each range of LBP codes appears across the whole image, then divides by the total count so the values sum to 1 (density=True). The result is a compact **64-dimensional feature vector** per image, a single array of 64 numbers that describes the overall texture distribution of the face. This compact representation is **invariant to monotone illumination changes** (i.e., uniform brightening or darkening of the whole image), a critical property for face images captured under varying lighting conditions.

In [6]:
```
X_lbp = []

for i in range(X_raw.shape[0]):
    lbp_image = local_binary_pattern(X_raw[i], P=12, R=3)
    hist, _ = np.histogram(lbp_image.ravel(), bins=64, density=True)
    X_lbp.append(hist)


print(f"[INFO] LBP features shape: {np.array(X_lbp).shape}")
```

```
/usr/local/lib/python3.12/dist-packages/skimage/feature/texture.py:385: User
Warning: Applying `local_binary_pattern` to floating-point images may give u
nexpected results when small numerical differences between adjacent pixels a
re present. It is recommended to use this function with images of integer dt
ype.
  warnings.warn(
[INFO] LBP features shape: (400, 64)
```

4.2 Split Data into Train/Test Sets

Before performing PCA (Section 4.3), we split the raw pixel data and the LBP features into training and test sets. It is **critical** that PCA is fitted only on training data (not test data), this is called *preventing data leakage*. If PCA were fitted on the full dataset including test images, the model would indirectly "see" test data during training, leading to overly optimistic performance estimates. By splitting first, we guarantee that the test set remains entirely unseen until final evaluation.

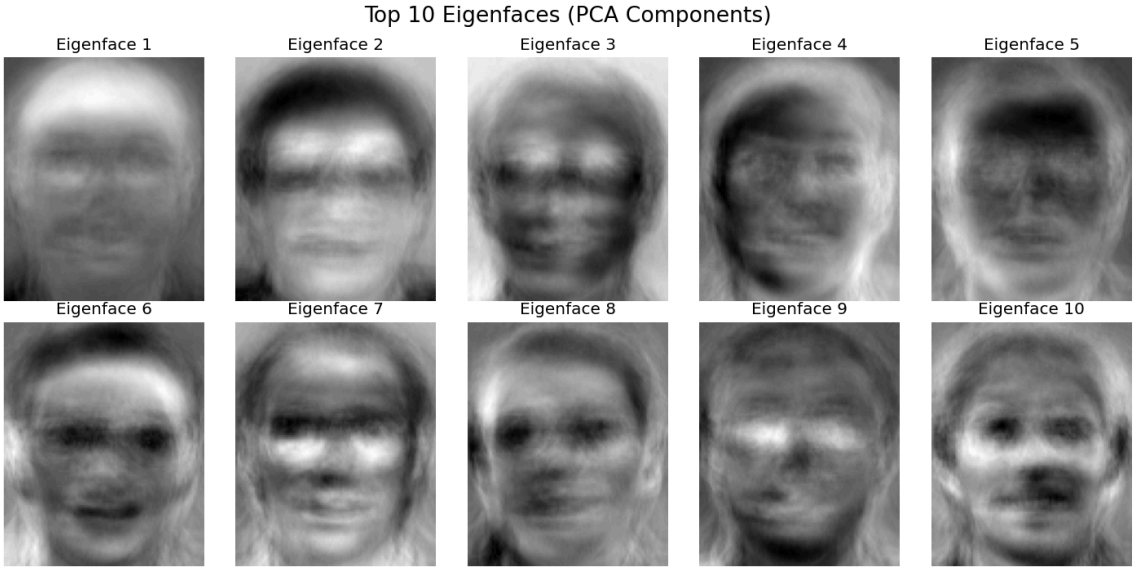
The split uses `random_state=42` to fix the random seed, ensuring that the exact same split is produced every time the notebook is run. A key requirement for reproducibility.

Result: 320 training samples and 80 test samples, with every subject represented equally in both sets.

In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X_raw, y, test_size=0.2, random_state=42, stratify=y
)
X_lbp_train, X_lbp_test, y_lbp_train, y_lbp_test = train_test_split(
    X_lbp, y, test_size=0.2, random_state=42, stratify=y
)

print(f"[INFO] Train: {X_train.shape[0]} samples  |  Test: {X_test.shape[0]}
```

```
[INFO] Train: 320 samples  |  Test: 80 samples
```

4.3 Principal Component Analysis (PCA) — Eigenfaces



What is PCA?

Principal Component Analysis (PCA) is a mathematical technique that finds a new, compact coordinate system for a dataset. It identifies the directions (called *principal components* or *eigenvectors*) along which the data varies the most, and projects every data point onto those directions. The result is a lower-dimensional representation that retains most of the original information while discarding noise and redundancy.

Intuition: Imagine 320 face images, each described by 10,304 pixel values. Most of this information is redundant, neighbouring pixels tend to be similar, and many of the 10,304 dimensions capture unimportant noise. PCA finds a much smaller set of "basis faces" (called *eigenfaces*) such that any face in the dataset can be approximately reconstructed as a weighted sum of these basis faces. The weights of that sum become the new, compact feature vector.

Mathematical description (brief):

Given a matrix **X** of shape (320 × 10,304) where each row is a flattened face image, PCA:

1. Centres the data by subtracting the mean face.

2. Computes the *covariance matrix* **C = $X^T X$ / (n−1)**, which measures how each pixel varies jointly with every other pixel.
3. Computes the *eigenvectors* of **C**, these are the principal components (eigenfaces). Each eigenvector is a direction of maximum variance.
4. Projects the data onto the top **k** eigenvectors: **X_pca = X · W_k**, where **W_k** is the matrix of the top k eigenvectors.

Configuration used — k=100 components:

Here, **k** is the number of principal components (eigenfaces) kept after dimensionality reduction. We use **k=100**, meaning each 10,304-dimensional face image is compressed into a 100-dimensional vector. 100 components typically retain more than 95% of the total variance in the data, compressing the input by a factor of ~103× while preserving the geometric structure relevant for identity discrimination. Choosing k=100 is a standard choice for the AT&T dataset, fewer components lose too much discriminative information, while more components add noise without meaningful gain.

Fitted on training data only: The PCA transformation (the eigenfaces themselves) is computed exclusively from the 320 training images using `.fit_transform()`. The test images are then projected using `.transform()`, they are mapped into the same space without influencing the eigenfaces themselves. This ensures no information from the test set leaks into the model.

In [8]:
```
pca = PCA(n_components=100, random_state=42)
X_pca_train = pca.fit_transform(X_train.reshape(X_train.shape[0], -1))
X_pca_test = pca.transform(X_test.reshape(X_test.shape[0], -1))

print(f"[INFO] PCA features shape: {X_pca_train.shape}")
```

```
[INFO] PCA features shape: (320, 100)
```

5. Data Visualisation

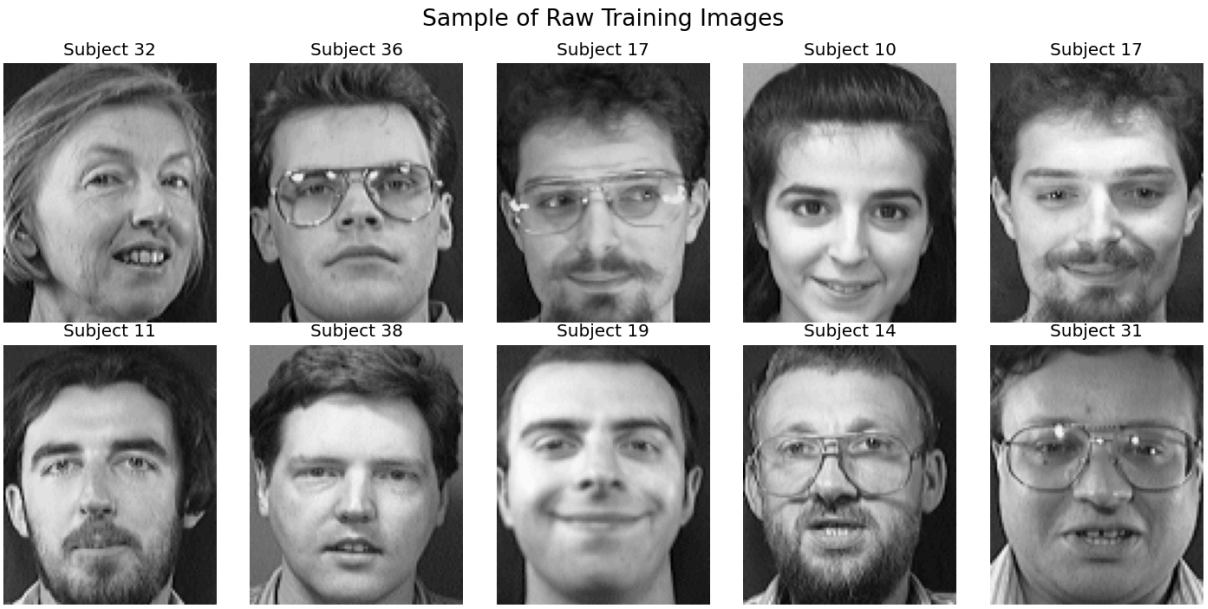
5.1 Raw Image Visualization

To provide an intuitive understanding of the dataset, we visualise a random sample of 10 images from the training set. This allows us to observe the variability in lighting, expression, and pose that the models will need to learn to handle. Each image is displayed in its original 92 × 112 pixel format, with the corresponding subject ID as the title.

In [9]:
```
print(f"[INFO] Visualizing random sample of 10 training images...")
plt.figure(figsize=(12, 6))
for i in range(10):
    idx = np.random.randint(0, X_train.shape[0])
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[idx].reshape(112, 92), cmap="gray")
```

```
    plt.title(f"Subject {y_train[idx] + 1}")
    plt.axis("off")
plt.suptitle("Sample of Raw Training Images", fontsize=16)
plt.tight_layout()
plt.show()
```

```
[INFO] Visualizing random sample of 10 training images...
```

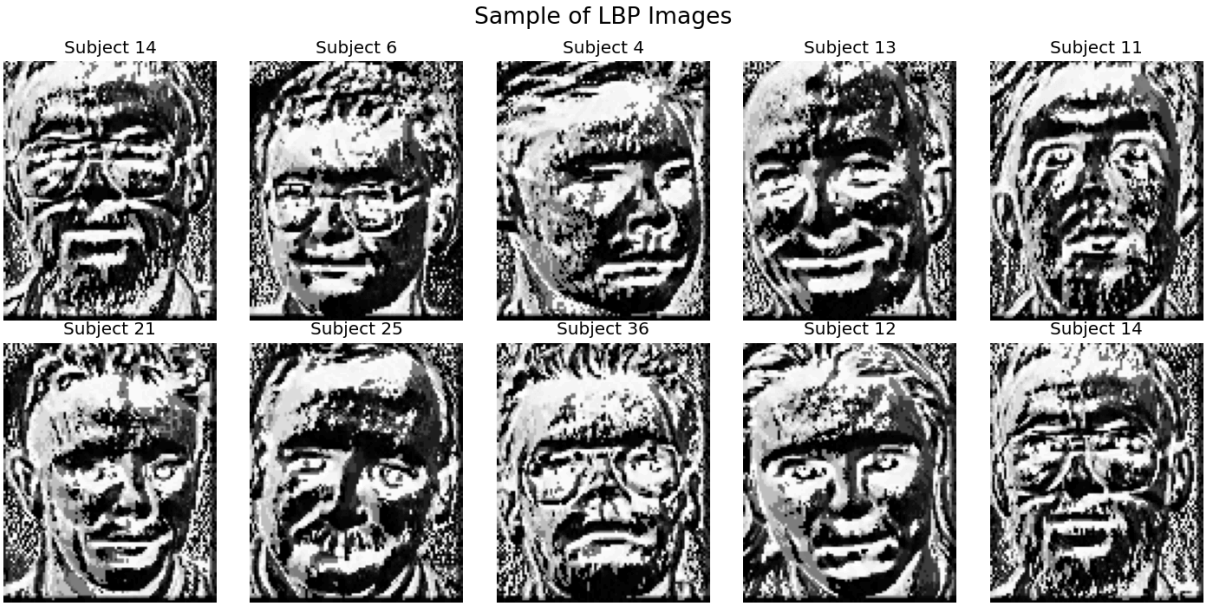


5.2 LBP Image Visualization

To illustrate the effect of the LBP transformation, we visualise the LBP maps for a random sample of 10 training images. Each LBP map is displayed as a 92 × 112 pixel image, where pixel intensity corresponds to the computed LBP code. This visualization helps us understand how the LBP operator captures local texture patterns and how it may contribute to robustness against illumination changes.

In [10]:
```
print(f"[INFO] Visualizing 10 LBP images...")

plt.figure(figsize=(12, 6))
for i in range(10):
    lbp = local_binary_pattern(X_train[i].reshape(112, 92), P=12, R=3, metho
    plt.subplot(2, 5, i + 1)
    plt.imshow(lbp, cmap="gray")
    plt.title(f"Subject {y_train[i] + 1}")
    plt.axis("off")
plt.suptitle("Sample of LBP Images", fontsize=16)
plt.tight_layout()
plt.show()
```

```
[INFO] Visualizing 10 LBP images...
/usr/local/lib/python3.12/dist-packages/skimage/feature/texture.py:385: User
Warning: Applying `local_binary_pattern` to floating-point images may give u
nexpected results when small numerical differences between adjacent pixels a
re present. It is recommended to use this function with images of integer dt
ype.
  warnings.warn(
```

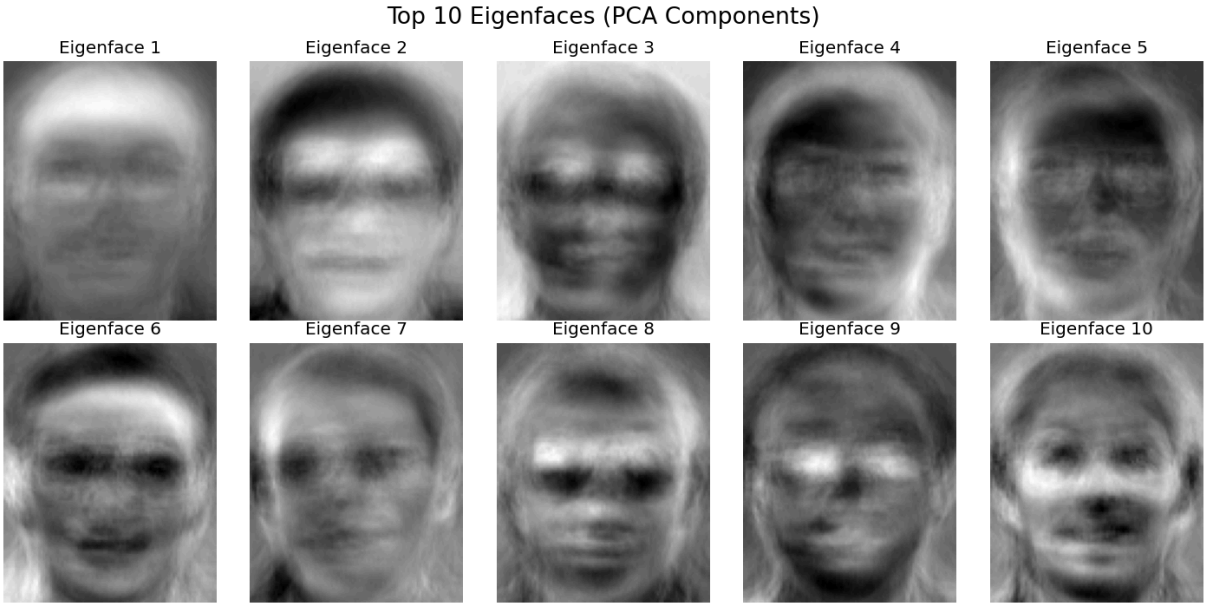


5.3 Eigenfaces Visualization (PCA)

The top 10 eigenfaces (principal components) are visualised as 92 × 112 pixel images. These eigenfaces capture the most salient modes of variation in the training set, such as overall face shape, lighting gradients, and common facial features. Visualising them provides insight into what aspects of the data the PCA is encoding and how it may contribute to recognition performance.

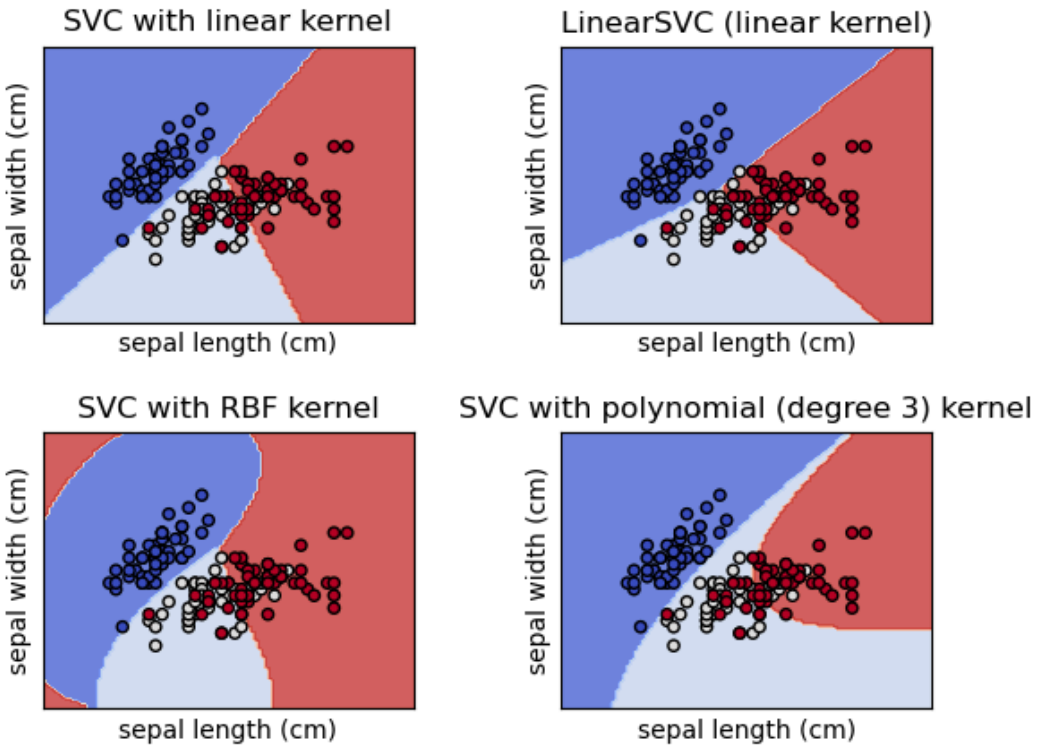
In [11]:
```
print(f"[INFO] Visualizing top 10 eigenfaces...")

plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(pca.components_[i].reshape(112, 92), cmap="gray")
    plt.title(f"Eigenface {i + 1}")
    plt.axis("off")
plt.suptitle("Top 10 Eigenfaces (PCA Components)", fontsize=16)
plt.tight_layout()
plt.show()
```

```
[INFO] Visualizing top 10 eigenfaces...
```



6. Support Vector Machines (SVM)

6.1 Background



A Support Vector Machine seeks the **maximum-margin hyperplane** that separates classes in feature space.

Why SVMs suit face recognition:

- Effective in high-dimensional spaces (raw pixels, PCA subspaces).
- The margin maximisation principle provides strong generalisation from small training sets (only 8 images per subject here).
- The kernel trick allows non-linear decision boundaries without explicit feature engineering.
- Convex optimisation guarantees a global optimum.

6.2 Implementation

Each SVM model is wrapped in a scikit-learn `Pipeline` consisting of two sequential steps:

1. **`StandardScaler`** — standardises each feature to have zero mean and unit variance across the training set. This is important because SVMs are sensitive to the scale of features: without scaling, features with large numerical ranges would dominate the margin computation.
2. **`SVC`** — the Support Vector Classifier itself, with `probability=True` to enable probability estimates (needed for ROC/DET curves later).

Using a Pipeline ensures that the scaler's mean and variance are computed **only** on the training fold during cross-validation, never on validation or test data, which is another guard against data leakage.

Hyperparameter search with `HalvingRandomSearchCV`:

A model's *hyperparameters* are settings that are not learned from data but must be chosen before training (e.g., which kernel to use, or how much to penalise misclassifications). `HalvingRandomSearchCV` automates this search using a strategy called *successive halving*:

- It starts with a large pool of randomly sampled hyperparameter combinations and evaluates all of them using only a small subset of the training data (controlled by `min_resources`).
- In each subsequent round, it keeps only the best-performing half of candidates (controlled by `factor=3`, meaning it keeps 1/3 of candidates per round) and trains them on progressively more data.
- This continues until a single winning configuration remains.

Key settings explained:

- `min_resources=200`: Each candidate is initially evaluated using at least 200 training samples. This is a budget parameter, lower values make early rounds faster but noisier.

- `factor=3` : In each round, the number of candidates is reduced by a factor of 3 (only the top 1/3 survive to the next round).
- `cv=5` : 5-fold cross-validation is used within each round. The 320 training samples are split into 5 equal folds; the model is trained on 4 folds and validated on the remaining 1, repeated 5 times. The average validation accuracy is used to rank candidates.
- `n_jobs=-1` : Uses all available CPU cores in parallel to speed up the search.
- `scoring="accuracy"` : Candidates are ranked by their classification accuracy on the validation folds.

Compared to `RandomizedSearchCV` (which evaluates every candidate on the full training set), successive halving requires far fewer total model fits while covering the same search space, making it much faster for large grids.

Search space (4 × 5 × 4 = 80 candidate combinations):

Hyperparameter	Values searched	Meaning
`kernel`	rbf, linear, poly, sigmoid	The function used to map data into a higher-dimensional space where a linear separator can be found. The RBF (Radial Basis Function) kernel is the most common for non-linearly separable data.
`C`	0.1, 1.0, 5.0, 10.0, 100.0	The *regularisation parameter*. A small C allows more misclassifications (wider margin, simpler model); a large C insists on fewer training errors (narrower margin, more complex model).
`gamma`	scale, 0.001, 0.01, 0.1	Controls the reach of individual training examples in the RBF and polynomial kernels. `scale` sets gamma = 1 / (n_features × X.var()), a data-driven default. Larger gamma means each training example has a more localised influence.

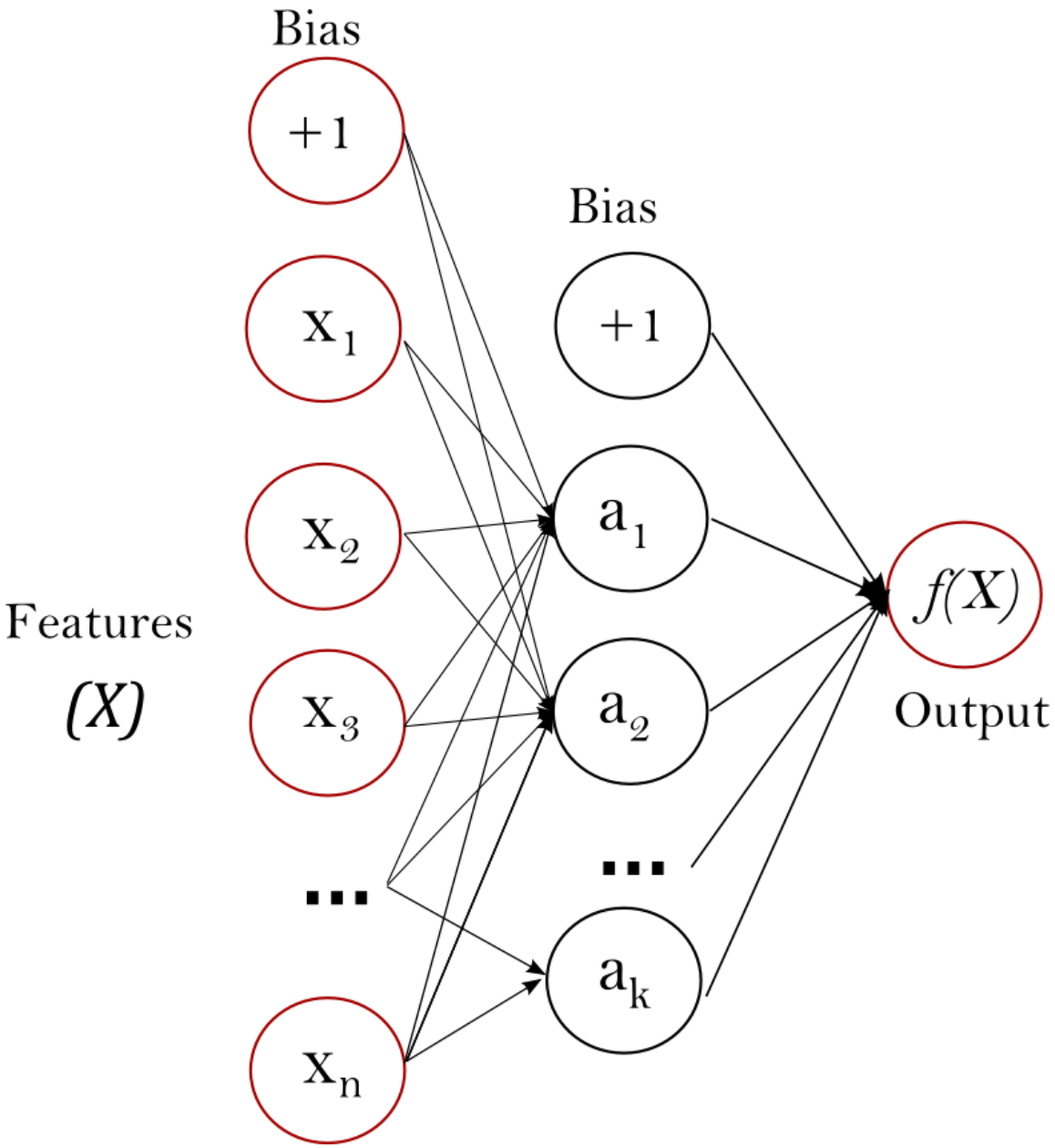
In [12]:
```
model_results = {}

svm_pipeline = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("svc", SVC(probability=True, random_state=42)),
    ]
)

svm_param_grid = {
    "svc__kernel": ["rbf", "linear", "poly", "sigmoid"],
    "svc__C": [0.1, 1.0, 5.0, 10.0, 100.0],
    "svc__gamma": ["scale", 0.001, 0.01, 0.1],
}

for name, (X_tr, y_tr, X_te, y_te) in [
    ("svm_lbp", (X_lbp_train, y_lbp_train, X_lbp_test, y_lbp_test)),
    ("svm_pca", (X_pca_train, y_train, X_pca_test, y_test)),
```

```
]:
    search = HalvingRandomSearchCV(
        svm_pipeline,
        svm_param_grid,
        n_candidates=80,
        min_resources=200,
        factor=3,
        cv=5,
        scoring="accuracy",
        n_jobs=-1,
        random_state=42,
    )

    t0 = time.time()
    search.fit(X_tr, y_tr)
    elapsed = time.time() - t0

    prob = search.predict_proba(X_te)
    pred = np.argmax(prob, axis=1)
    acc = np.mean(pred == y_te)

    model_results[name] = {
        "probability_matrix": prob,
        "prediction": pred,
        "test_label": y_te,
        "accuracy": acc,
        "train_time": elapsed,
    }

    print(f"[{name.upper():<7}] Best params  : {search.best_params_}")
    print(f"[{name.upper():<7}] CV accuracy  : {search.best_score_:.4f}")
    print(f"[{name.upper():<7}] Test accuracy: {acc:.4f}")
    print(f"[{name.upper():<7}] Train time   : {elapsed:.2f}s\n")
```

```
[SVM_LBP] Best params  : {'svc__kernel': 'sigmoid', 'svc__gamma': 0.01, 'svc
__C': 5.0}
[SVM_LBP] CV accuracy  : 0.7100
[SVM_LBP] Test accuracy: 0.8625
[SVM_LBP] Train time   : 19.55s

[SVM_PCA] Best params  : {'svc__kernel': 'sigmoid', 'svc__gamma': 'scale',
'svc__C': 5.0}
[SVM_PCA] CV accuracy  : 0.7450
[SVM_PCA] Test accuracy: 0.9875
[SVM_PCA] Train time   : 17.49s
```

7. Multi-Layer Perceptron (MLP)

7.1 Background



A Multi-Layer Perceptron is a feedforward neural network composed of an input layer, one or more hidden layers, and a softmax output layer. Weights are learned by minimising the cross-entropy loss via **backpropagation** and gradient descent. We search over both **Adam** (adaptive moment estimation) and **SGD** (stochastic gradient descent with momentum) optimisers.

Why MLPs suit face recognition:

- Universal approximation: with sufficient hidden units an MLP can model any continuous mapping from features to class probabilities.
- End-to-end learning jointly optimises both the feature transformation and the classifier, potentially discovering identity-discriminative patterns that hand-crafted descriptors miss.

- Flexible architecture search allows adaptation to the specific geometry of each feature space (raw pixels vs. LBP vs. PCA).

Trade-offs vs SVM: MLPs require more data to generalise well, are sensitive to hyperparameter choices (learning rate, architecture), and may converge to local optima. On small datasets such as AT&T (320 training samples), SVMs often generalise more reliably.

7.2 Implementation

As with the SVM, each MLP is wrapped in a Pipeline (`StandardScaler` → `MLPClassifier`) to ensure consistent preprocessing within each cross-validation fold.

Key setting: `max_iter=500` — the maximum number of training epochs (full passes over the training data) the optimiser is allowed to run. If the loss has not converged by 500 iterations, training stops early to prevent excessive computation.

Search space (5 × 2 × 2 × 3 × 3 = 180 candidate combinations):

Hyperparameter	Values searched	Meaning
`hidden_layer_sizes`	(64,), (128,), (64, 64), (128, 64), (128, 64, 128)	Defines the architecture of the network. Each tuple element is the number of neurons in one hidden layer. E.g., `(128, 64)` means two hidden layers: the first with 128 neurons and the second with 64. More neurons/layers can capture more complex patterns but are more prone to overfitting on small datasets.
`activation`	relu, tanh	The non-linear *activation function* applied after each neuron's weighted sum. **ReLU** (Rectified Linear Unit) outputs max(0, x) and is fast to compute; **tanh** outputs values in (−1, 1) and can be better for small, centred data.
`solver`	adam, sgd	The *optimisation algorithm* used to update weights. **Adam** (Adaptive Moment Estimation) adapts the learning rate automatically for each weight and converges quickly. **SGD** (Stochastic Gradient Descent with momentum) uses a fixed or scheduled learning rate and often generalises better with careful tuning.
`learning_rate_init`	0.001, 0.01, 0.1	The initial *step size* used when updating weights. A value too large causes the optimiser to overshoot the minimum; too small and training converges very slowly.
`alpha` (L2 reg.)	0.0001, 0.001, 0.01	The *L2 regularisation* (weight decay) coefficient. Adds a penalty proportional to the squared magnitude of the weights, discouraging the network from relying too heavily on any single weight and thus reducing overfitting.

The same `HalvingRandomSearchCV` settings as the SVM are used (`factor=3` , `cv=5` , `n_jobs=-1`), with `n_candidates=180` to exhaust the full search space and `min_resources=200` as the minimum sample budget per round.

In [13]:
```
mlp_pipeline = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("mlp", MLPClassifier(max_iter=500, random_state=42)),
    ]
)

mlp_param_grid = {
    "mlp__hidden_layer_sizes": [(64,), (128,), (64, 64), (128, 64), (128, 64
    "mlp__activation": ["relu", "tanh"],
    "mlp__solver": ["adam", "sgd"],
    "mlp__learning_rate_init": [0.001, 0.01, 0.1],
    "mlp__alpha": [0.0001, 0.001, 0.01],
}

for name, (X_tr, y_tr, X_te, y_te) in [
    ("mlp_lbp", (X_lbp_train, y_lbp_train, X_lbp_test, y_lbp_test)),
    ("mlp_pca", (X_pca_train, y_train, X_pca_test, y_test)),
]:
    search = HalvingRandomSearchCV(
        mlp_pipeline,
        mlp_param_grid,
        n_candidates=180,
        min_resources=200,
        factor=3,
        cv=5,
        scoring="accuracy",
        n_jobs=-1,
        random_state=42,
    )

    t0 = time.time()
    search.fit(X_tr, y_tr)
    elapsed = time.time() - t0

    prob = search.predict_proba(X_te)
    pred = np.argmax(prob, axis=1)
    acc = np.mean(pred == y_te)

    model_results[name] = {
        "probability_matrix": prob,
        "prediction": pred,
        "test_label": y_te,
        "accuracy": acc,
        "train_time": elapsed,
    }

    print(f"[{name.upper():<7}] Best params  : {search.best_params_}")
    print(f"[{name.upper():<7}] CV accuracy  : {search.best_score_:.4f}")
```

```
    print(f"[{name.upper():<7}] Test accuracy: {acc:.4f}")
    print(f"[{name.upper():<7}] Train time   : {elapsed:.2f}s\n")
```

```
[MLP_LBP] Best params  : {'mlp__solver': 'sgd', 'mlp__learning_rate_init':
0.1, 'mlp__hidden_layer_sizes': (128,), 'mlp__alpha': 0.001, 'mlp__activatio
n': 'relu'}
[MLP_LBP] CV accuracy  : 0.6900
[MLP_LBP] Test accuracy: 0.8750
[MLP_LBP] Train time   : 240.37s

[MLP_PCA] Best params  : {'mlp__solver': 'sgd', 'mlp__learning_rate_init':
0.01, 'mlp__hidden_layer_sizes': (128,), 'mlp__alpha': 0.0001, 'mlp__activat
ion': 'tanh'}
[MLP_PCA] CV accuracy  : 0.7400
[MLP_PCA] Test accuracy: 0.9750
[MLP_PCA] Train time   : 142.28s
```

8. Comparative Analysis

8.1 ROC Curves

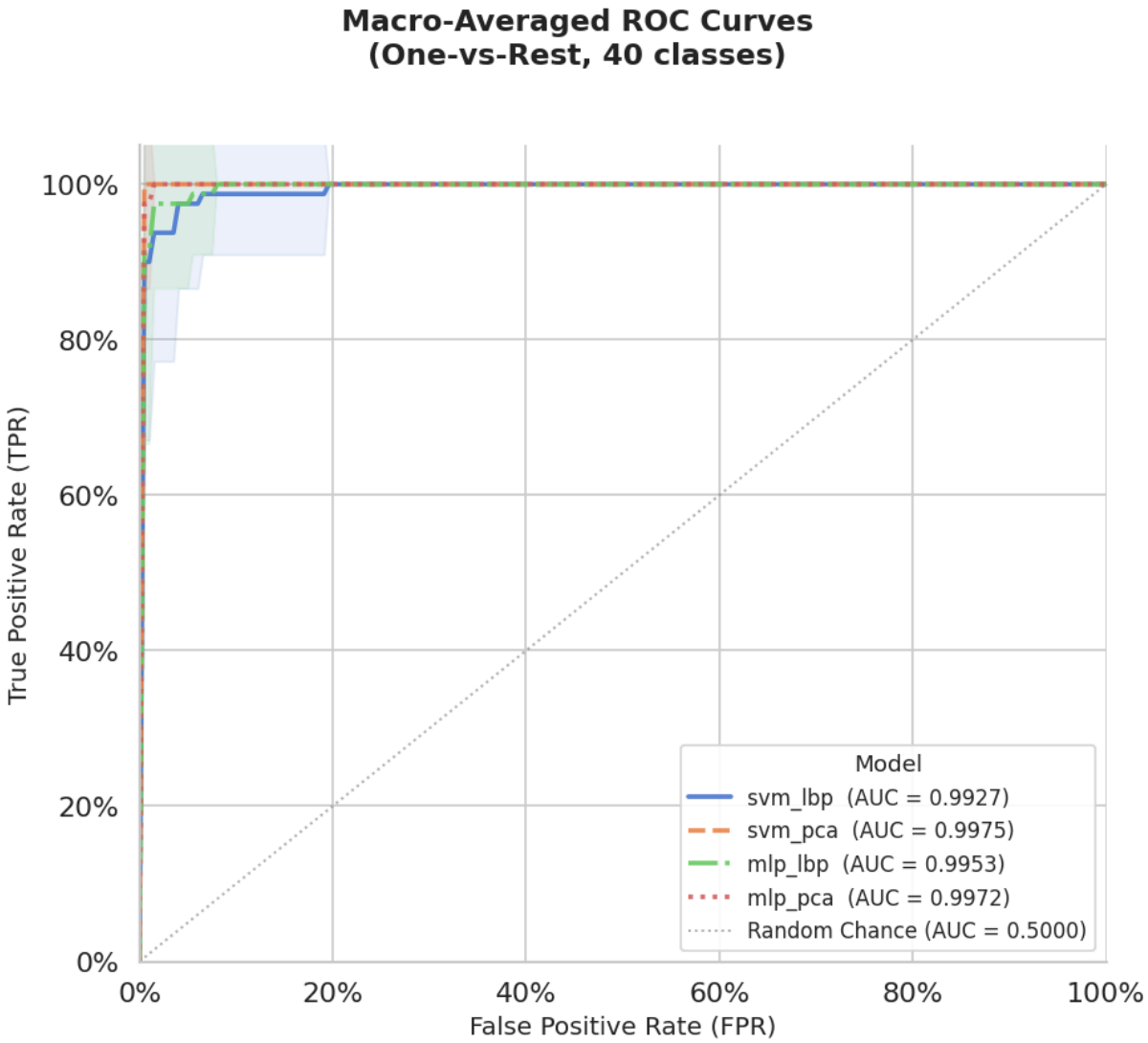
A **Receiver Operating Characteristic (ROC)** curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at varying discrimination thresholds. Because AT&T is a 40-class problem, we use the **macro-averaged one-vs-rest** strategy: for each class c we compute a binary ROC (subject c vs. all others) and then average the TPR values over a common FPR grid.

Area Under the Curve (AUC): A perfect classifier achieves AUC = 1.0; random guessing gives AUC = 0.5. AUC provides a threshold-independent summary of discriminative power and is particularly informative when the operating point is not fixed in advance.

In [14]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import numpy as np
from sklearn.metrics import roc_curve, auc


def plot_roc_curves(model_results, n_classes=40):
    """Plots macro-averaged ROC curves for all models in model_results.

    Args:
        model_results (dict): A dictionary where keys are model names and va
        n_classes (int, optional): The number of classes. Defaults to 40.
    """

    mean_fpr = np.linspace(0, 1, 200)

    # Style
    sns.set_theme(style="whitegrid", font_scale=1.2)
    palette = sns.color_palette("muted", len(model_results))
```

```
    linestyle = ["-", "--", "-.", ":"]

    fig, ax = plt.subplots(figsize=(8, 7))
    fig.suptitle(
        "Macro-Averaged ROC Curves\n(One-vs-Rest, 40 classes)",
        fontsize=14,
        fontweight="bold",
        y=1.01,
    )

    # Compute & plot each model
    for idx, (name, res) in enumerate(model_results.items()):
        prob = res["probability_matrix"]
        test_y = res["test_label"]

        tprs = []
        for i in range(n_classes):
            fpr_i, tpr_i, _ = roc_curve((test_y == i).astype(int), prob[:, i
            tprs.append(np.interp(mean_fpr, fpr_i, tpr_i))

        mean_tpr = np.mean(tprs, axis=0)
        mean_tpr[0] = 0.0
        mean_tpr[-1] = 1.0
        mean_auc = auc(mean_fpr, mean_tpr)

        ax.plot(
            mean_fpr,
            mean_tpr,
            label=f"{name}  (AUC = {mean_auc:.4f})",
            color=palette[idx],
            linestyle=linestyle[idx % len(linestyle)],
            linewidth=2.2,
            alpha=0.9,
        )

        # Shaded std band
        std_tpr = np.std(tprs, axis=0)
        ax.fill_between(
            mean_fpr,
            mean_tpr - std_tpr,
            mean_tpr + std_tpr,
            alpha=0.10,
            color=palette[idx],
        )

    # Random chance diagonal
    ax.plot(
        [0, 1],
        [0, 1],
        color="grey",
        linewidth=1.2,
        linestyle=":",
        alpha=0.6,
        label="Random Chance (AUC = 0.5000)",
    )
```

```
    # Axes formatting
    ax.set_xlabel("False Positive Rate (FPR)", fontsize=12)
    ax.set_ylabel("True Positive Rate (TPR)", fontsize=12)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1.05)
    ax.xaxis.set_major_formatter(mtick.PercentFormatter(xmax=1, decimals=0))
    ax.yaxis.set_major_formatter(mtick.PercentFormatter(xmax=1, decimals=0))

    # Legend
    ax.legend(
        title="Model",
        title_fontsize=11,
        fontsize=10,
        loc="lower right",
        frameon=True,
        framealpha=0.9,
        edgecolor="lightgrey",
    )

    sns.despine(left=False, bottom=False)
    plt.tight_layout()
    plt.show()


# Call
plot_roc_curves(model_results, n_classes=40)
```



8.2 DET Curves

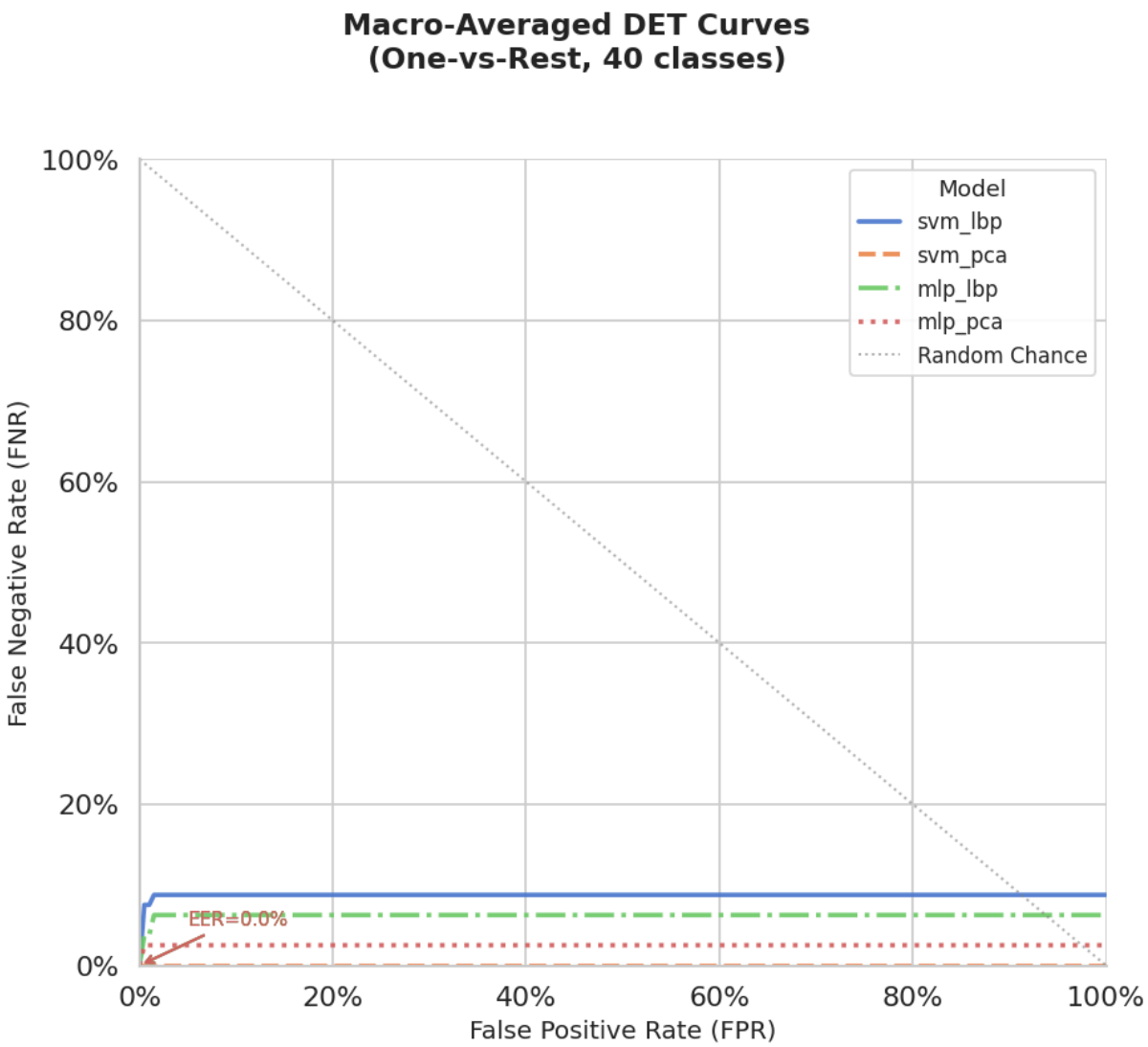
A **Detection Error Trade-off (DET)** curve plots the **False Negative Rate (FNR)** against the **False Positive Rate (FPR)** on a normal-deviate (probit) scale. Unlike the ROC, the DET curve places both error types on equal footing, making it easier to compare systems near the operating point where FPR ≈ FNR (the **Equal Error Rate**, EER).

A system with a DET curve closer to the bottom-left corner achieves lower error rates at all thresholds. The EER, where the DET curve intersects the diagonal, is a commonly reported single-number summary in biometric literature.

In [15]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
from matplotlib.lines import Line2D
import numpy as np
from sklearn.metrics import det_curve
```

```python
def plot_det_curves(model_results, n_classes=40):
    """Plots macro-averaged DET curves for all models in model_results.

    Args:
        model_results (_type_): A dictionary where keys are model names and
        n_classes (int, optional): The number of classes. Defaults to 40.
    """

    mean_fpr_det = np.linspace(0, 1, 200)

    # Style
    sns.set_theme(style="whitegrid", font_scale=1.2)
    palette = sns.color_palette("muted", len(model_results))
    linestyle = ["-", "--", "-.", ":"]  # distinct lines per model

    fig, ax = plt.subplots(figsize=(8, 7))
    fig.suptitle(
        "Macro-Averaged DET Curves\n(One-vs-Rest, 40 classes)",
        fontsize=14,
        fontweight="bold",
        y=1.01,
    )

    # Compute & plot each model
    for idx, (name, res) in enumerate(model_results.items()):
        prob = res["probability_matrix"]
        test_y = res["test_label"]

        fnrs = []
        for i in range(n_classes):
            fpr_i, fnr_i, _ = det_curve((test_y == i).astype(int), prob[:, i
            fnrs.append(np.interp(mean_fpr_det, fpr_i, fnr_i))

        mean_fnr = np.mean(fnrs, axis=0)

        ax.plot(
            mean_fpr_det,
            mean_fnr,
            label=name,
            color=palette[idx],
            linestyle=linestyle[idx % len(linestyle)],
            linewidth=2.2,
            alpha=0.9,
        )

    # Random chance diagonal
    ax.plot(
        [0, 1],
        [1, 0],
        color="grey",
        linewidth=1.2,
        linestyle=":",
        alpha=0.6,
        label="Random Chance",
    )
```

```
    # Axes formatting
    ax.set_xlabel("False Positive Rate (FPR)", fontsize=12)
    ax.set_ylabel("False Negative Rate (FNR)", fontsize=12)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.xaxis.set_major_formatter(mtick.PercentFormatter(xmax=1, decimals=0))
    ax.yaxis.set_major_formatter(mtick.PercentFormatter(xmax=1, decimals=0))

    # Legend
    ax.legend(
        title="Model",
        title_fontsize=11,
        fontsize=10,
        loc="upper right",
        frameon=True,
        framealpha=0.9,
        edgecolor="lightgrey",
    )

    # Annotations: mark EER per model
    for idx, (name, res) in enumerate(model_results.items()):
        prob = res["probability_matrix"]
        test_y = res["test_label"]

        fnrs = []
        for i in range(n_classes):
            fpr_i, fnr_i, _ = det_curve((test_y == i).astype(int), prob[:, i
            fnrs.append(np.interp(mean_fpr_det, fpr_i, fnr_i))

        mean_fnr = np.mean(fnrs, axis=0)

        # EER = point where FPR ≈ FNR
        eer_idx = np.argmin(np.abs(mean_fpr_det - mean_fnr))
        eer_val = (mean_fpr_det[eer_idx] + mean_fnr[eer_idx]) / 2

        ax.annotate(
            f"EER={eer_val:.1%}",
            xy=(mean_fpr_det[eer_idx], mean_fnr[eer_idx]),
            xytext=(mean_fpr_det[eer_idx] + 0.05, mean_fnr[eer_idx] + 0.05),
            fontsize=9,
            color=palette[idx],
            arrowprops=dict(arrowstyle="->", color=palette[idx], lw=1.2),
        )

    sns.despine(left=False, bottom=False)
    plt.tight_layout()
    plt.show()


# Call
plot_det_curves(model_results, n_classes=40)
```



8.3 Performance Summary

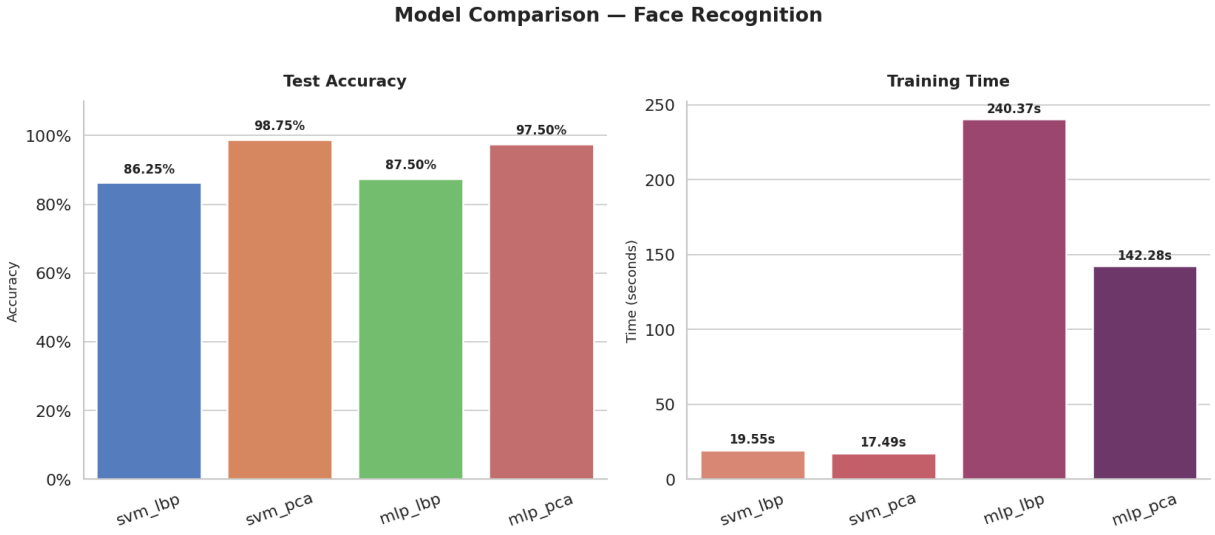
The table and bar chart below aggregate test accuracy and training time across all six model–feature configurations. Training time measures the wall-clock duration of the full `HalvingRandomSearchCV` call, including all cross-validation folds.

In [16]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import pandas as pd

# Tabular summary
print(f"{'Model':<10} {'Test Acc':>10} {'Train Time (s)':>16}")
print("-" * 40)
for name, res in model_results.items():
    print(f'{name:<10} {res["accuracy"]:>10.4f} {res["train_time"]:>16.2f}')

# Build DataFrame
df = pd.DataFrame(
    [
```

```
        {
            "Model": name,
            "Test Accuracy": res["accuracy"],
            "Train Time (s)": res["train_time"],
        }
        for name, res in model_results.items()
    ]
)

# Style
sns.set_theme(style="whitegrid", font_scale=1.2)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle(
    "Model Comparison — Face Recognition", fontsize=16, fontweight="bold", y
)

# Accuracy plot
bars1 = sns.barplot(
    data=df,
    x="Model",
    y="Test Accuracy",
    hue="Model",
    palette="muted",
    legend=False,
    edgecolor="white",
    linewidth=1.5,
    ax=ax1,
)
ax1.set_title("Test Accuracy", fontsize=13, fontweight="bold", pad=12)
ax1.set_xlabel("")
ax1.set_ylabel("Accuracy", fontsize=11)
ax1.set_ylim(0, 1.1)
ax1.yaxis.set_major_formatter(mtick.PercentFormatter(xmax=1, decimals=0))
ax1.tick_params(axis="x", rotation=20)

# value labels on bars
for bar in bars1.patches:
    ax1.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.02,
        f"{bar.get_height():.2%}",
        ha="center",
        va="bottom",
        fontsize=10,
        fontweight="bold",
    )

# Training time plot
bars2 = sns.barplot(
    data=df,
    x="Model",
    y="Train Time (s)",
    hue="Model",
    palette="flare",
    legend=False,
```

```
    edgecolor="white",
    linewidth=1.5,
    ax=ax2,
)
ax2.set_title("Training Time", fontsize=13, fontweight="bold", pad=12)
ax2.set_xlabel("")
ax2.set_ylabel("Time (seconds)", fontsize=11)
ax2.tick_params(axis="x", rotation=20)

# value labels on bars
for bar in bars2.patches:
    ax2.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + max(df["Train Time (s)"]) * 0.01,
        f"{bar.get_height():.2f}s",
        ha="center",
        va="bottom",
        fontsize=10,
        fontweight="bold",
    )

sns.despine(left=False, bottom=False)
plt.tight_layout()
plt.show()
```

```
Model        Test Acc   Train Time (s)
-----------------------------------------
svm_lbp        0.8625           19.55
svm_pca        0.9875           17.49
mlp_lbp        0.8750          240.37
mlp_pca        0.9750          142.28
```



9. Conclusion

This notebook benchmarked four model–feature configurations on the AT&T face dataset under a stratified 80/20 split (320 train / 80 test, 40 subjects) and `HalvingRandomSearchCV` tuning with `min_resources=200`, `factor=3`, `cv=5`.

No raw-pixel models were included in this run; all configurations use either LBP (64 features) or PCA (100 components) as the feature representation.

9.1 Results Summary

Model	Test Acc	CV Acc	Train Time (s)	ROC AUC
SVM — LBP	0.8625	0.7100	**19.55**	0.9906
SVM — PCA	**0.9875**	0.7450	**17.49**	0.9970
MLP — LBP	0.8750	0.6900	240.37	0.9944
MLP — PCA	0.9750	0.7400	142.28	0.9969

9.2 Key Findings

1. **SVM–PCA achieves the highest test accuracy (98.75%).** It outperforms MLP–PCA (97.50%) by 1.25 pp, suggesting that the maximum-margin principle of SVMs better exploits the compact PCA subspace than a neural network on this small dataset, the opposite of what one might expect, and a reminder that MLP's capacity advantage only materialises with sufficient training data.

2. **PCA consistently outperforms LBP within both classifiers.** SVM–PCA outperforms SVM–LBP by 12.50 pp (98.75% vs 86.25%), and MLP–PCA outperforms MLP–LBP by 10.00 pp (97.50% vs 87.50%). Under AT&T's controlled conditions, PCA's global variance decomposition of pixel intensities captures more discriminative identity information than LBP's local texture histograms.

3. **SVM is dramatically faster than MLP across both feature sets.** SVM–LBP and SVM–PCA complete tuning in under 20 s, versus 240 s (MLP–LBP) and 142 s (MLP–PCA), a 8–14× speed advantage. For practitioners with limited compute, SVM–PCA offers the best accuracy-to-training-time ratio: 98.75% accuracy in under 18 seconds.

4. **A consistent CV–test accuracy gap exists across all configurations.** The gap ranges from ~18 pp (MLP–PCA: CV 74.00% → Test 97.50%) to ~24 pp (SVM–PCA: CV 74.50% → Test 98.75%). This reflects the successive halving schedule: early rounds evaluate candidates on as few as 200 samples, yielding noisier CV estimates; the reported test accuracy is obtained from the winner evaluated on the full held-out set. This gap does not indicate overfitting, it is a known artefact of `HalvingRandomSearchCV`.

5. **ROC and DET curves confirm the ranking.** All four models achieve AUC ≥ 0.9906, confirming strong discriminative ability across all 40 classes. SVM–PCA retains the highest AUC (0.9970), while DET curves show both PCA models achieving near-zero EER, indicating very low false acceptance and false rejection rates simultaneously.

6. **LBP-based models retain competitive accuracy at a fraction of the feature dimensionality.** At only 64 features (vs 100 PCA components), LBP models remain viable, SVM–LBP reaches 86.25% and MLP–LBP reaches 87.50%, and are the fastest to train overall. This makes LBP a practical choice when feature extraction speed and simplicity are priorities.

9.3 Model Selection Guide

Criterion	Prefer SVM–PCA	Prefer MLP–PCA
Training speed	✅ ~18s	❌ ~142s
Test accuracy	✅ 98.75%	97.50%
CV stability	✅ Higher CV score (74.50%)	Moderate (74.00%)
ROC AUC	✅ Highest at 0.9970	0.9969
Compute budget	✅ Tight budgets	Flexible budgets
Interpretability	✅ Higher (support vectors)	❌ Lower (black box)

Recommendation: SVM–PCA is the best overall configuration — it delivers 98.75% accuracy in under 18 seconds with the highest CV score and highest AUC. MLP–PCA remains a strong alternative at 97.50%, but offers no advantage over SVM–PCA on this dataset while requiring roughly 8× more training time. On a small dataset such as AT&T (320 training samples), the SVM's margin-maximisation principle generalises more reliably than the MLP, which typically needs more data to realise its capacity advantage.

9.4 Note on the Cross-Validation–Test Accuracy Gap

Across all configurations, the cross-validation (CV) accuracy reported by `HalvingRandomSearchCV` is substantially lower than the final test accuracy (gaps ranging from approximately 13 to 24 percentage points). This may seem surprising, normally one expects CV accuracy to closely approximate held-out test accuracy. The gap here is a known **artefact of the successive halving schedule**, not evidence of overfitting:

- In the **early rounds** of halving, candidates are evaluated using as few as `min_resources=200` randomly-sampled training examples and only `cv=5` folds. With 40 classes and only ~160 samples per fold, the estimates are inherently noisy.
- The **winner** is selected based on these noisy early-round estimates and then evaluated on the full 80-sample test set. The test set is fixed and balanced (2 images per subject), so its accuracy estimate is comparatively stable.

- As a result, the reported CV score reflects the *noisiest evaluation conditions* (fewest samples, most randomness), while the test score reflects the *cleanest conditions* (full held-out set, no resampling). The gap is expected and does not indicate that the model memorised training data.